

Contest Platform with Live Leaderboard

Problem Definition, Requirement Analysis & Database Design (MongoDB + Redis)

TEAM

Upayan Mazumder, 24BDS0367

Aditya Rawat, 24BCE2992

Shloak Sinha, 24BDS0378

DOCUMENT

Review 1 Report

Primary database: MongoDB

Supporting store: Redis

01 Problem Statement, Objectives & Scope

A contest platform runs timed contests where users, individually or in teams, attempt problems and are ranked on a live leaderboard. The platform has to manage **users, contests, problems, teams, submissions** and a continuously updated leaderboard. Problems are not uniform: an MCQ problem needs options and a correct answer, a coding problem needs test cases and constraints, a subjective problem needs a rubric and a word limit. Contests bundle a changing subset of these problems, and every submission made during a live contest is an independent write that must be queryable both by problem and by user.

Objectives

1. Model users, contests, problems, teams and submissions as MongoDB collections.
2. Support MCQ, coding and subjective problems in a single problems collection.
3. Define the reference relationships between collections and justify each one.
4. Describe, conceptually, how Redis supports the live leaderboard, sessions and rate limiting.
5. Build a small sample dataset that demonstrates the schema working end-to-end.

Scope

The platform covers contest creation, solo and team participation, MCQ, coding and subjective problems, submission storage and a live leaderboard. The judge that actually runs and checks code submissions is stubbed rather than a full execution sandbox, since the focus is the database design, not building a compiler service. Payments, proctoring and plagiarism checks are out of scope.

02 Requirement Specification

Type	Requirement
Functional	User registration and account management
Functional	Contest creation and participation
Functional	Support for MCQ, coding and subjective problems
Functional	Submission storage and status tracking
Functional	Team formation and membership
Functional	Leaderboard support
Non-functional	Flexible schema for heterogeneous problem types
Non-functional	Efficient retrieval of contest and submission data
Non-functional	Support for concurrent contest activity

03 Why MongoDB?

The problems collection is the main justification. MCQ, coding and subjective problems share a few common fields but diverge beyond that:

```
MCQ      → options, correctAnswer
Coding   → inputFormat, constraints, testCases
Subjective → wordLimit, evaluationRubric
```

All three fit in one collection because MongoDB does not require every document to share the same shape. A type field is enough for the application to tell them apart. Adding a fourth problem type later just means new documents, not a schema migration.

Situation	Relational	MongoDB
Storing 3 problem types	Separate tables, or one table with many nullable columns	One collection, one shape per type

Situation	Relational	MongoDB
Adding a 4th type	Schema migration	New documents, no migration
Problems in a contest	Join across tables	Single query filtered by contestId

Redis is used only for state that changes constantly and does not need to be durable: the live leaderboard (a sorted set per contest), active sessions (TTL keys), and submission rate limiting (TTL counters). MongoDB stays the single source of truth; Redis can always be rebuilt from it.

04 Database Design

Five collections cover the requirements above. A ref field holds another document's `_id`.

users _id name email passwordHash role // participant admin createdAt	contests _id title description startTime, endTime status // upcoming live ended createdBy // ref → users problemIds // ref → problems participants // ref → users/teams createdAt
problems _id contestId // ref → contests type // mcq coding subjective title, description difficulty, points [type-specific fields]	submissions _id contestId // ref → contests problemId // ref → problems submittedBy // ref → user/team answer status // pending correct incorrect partial score submittedAt
teams _id name memberIds // ref → users createdAt	

Relationships

```

User      → Contest
Contest   → Problems
User/Team → Submission
Submission → Problem
Team      → Users

```

All of these are **references**, not embedded documents. Problems and submissions get queried on their own (a user's submission history, or every problem in a contest), so nesting them inside a parent would just duplicate data. The one exception is a contest's participant list, kept as a small array of references since it's always read along with the contest.

ER-to-NoSQL mapping

ER concept	NoSQL representation
Entity	Collection (users, contests, problems, submissions, teams)
Attribute	Document field
1:N relationship (Contest to Problem)	Child document stores the parent's <code>_id</code>
M:N relationship (Team to User)	Array of references (memberIds)
Problem subtypes (MCQ/coding/subjective)	Polymorphic documents in one collection

05 Polymorphic Problem Example

Three documents from the same problems collection, same common fields, different type-specific fields.

type: "mcq"

```
{
  _id: ObjectId("p1"),
  contestId: ObjectId("c1"),
  type: "mcq",
  title: "Largest prime below 100",
  difficulty: "easy",
  points: 5,
  options: ["89", "91", "97", "99"],
  correctAnswer: 2
}
```

type: "coding"

```
{
  _id: ObjectId("p2"),
  contestId: ObjectId("c1"),
  type: "coding",
  title: "Two Sum",
  difficulty: "medium",
  points: 20,
  inputFormat: "array nums, int target",
  constraints: "2 ≤ nums.length ≤ 1e4",
  testCases: [{ input: "[2,7,11,15],9", output: "[0,1]" }]
}
```

type: "subjective"

```
{
  _id: ObjectId("p3"),
  contestId: ObjectId("c1"),
  type: "subjective",
  title: "Explain the CAP theorem",
  difficulty: "medium",
  points: 15,
  wordLimit: 200,
  evaluationRubric: "correctness, clarity, examples"
}
```

All three share `_id`, `contestId`, `type`, `title`, `difficulty` and `points`; each adds only the fields its type needs.

06 Sample Dataset

Just enough data to check the schema works. The full dataset comes later.

users

id	name	role
U1	Aditya Rawat	participant
U2	Shloak Sinha	participant
U3	Upayan Mazumder	admin

problems

id	type	points
P1	mcq	5
P2	coding	20
P3	subjective	15

teams

id	name	memberIds
T1	Byte Me	[U1, U2]

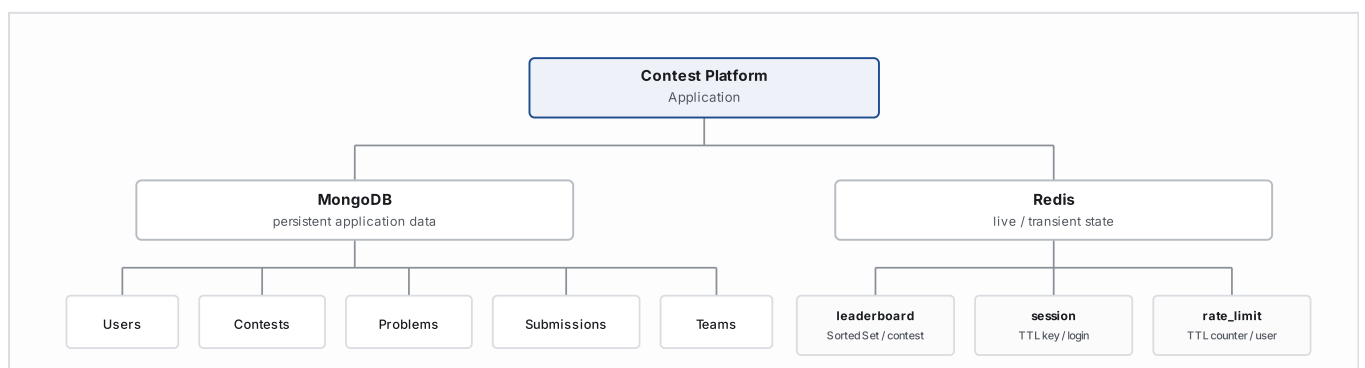
contests

id	title	problemIds
C1	Fall Sprint 2026	[P1, P2, P3]

submissions

id	problemId	submittedBy	status	score
S1	P1	{user, U1}	correct	5
S2	P2	{team, T1}	partial	12
S3	P3	{user, U2}	pending	-

07 Architecture



MongoDB holds the five persistent collections. Redis holds derived, transient state rebuilt from MongoDB data.